

**JEPPIAAR INSTITUTE OF TECHNOLOGY****(An Autonomous Institution)****"Self-Belief | Self-Discipline | Self-Respect"**

Kunnam, Sunguvarchatram, Sripurumbudur – 631 604.



DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND DATA SCIENCE



AIT301 - CRYPTOGRAPHY AND NETWORK SECURITY LABORATORY

NAME :

REG NO :

YEAR :

SEMESTER :

**DEPARTMENT OF ARTIFICIAL INTELLIGENCE ANDS DATA SCIENCE****BONAFIDE CERTIFICATE**

This is a certified bonafide record work of Mr./Ms _____ Reg. No. _____ submitted for the End Semester practical examination held on _____ in **AIT301 Cryptography and Network Security Laboratory** of V Semester / III Year during the year of 2026-2027.

Signature of the Lab In-charge

Head of the Department

Internal Examiner**External Examiner**

Date: _____

**JEPPIAAR INSTITUTE OF TECHNOLOGY****(An Autonomous Institution)****“Self-Belief | Self-Discipline | Self-Respect”**

Kunnam, Sunguvarchatram, Sriperumbudur – 631 604.



VISION AND MISSION OF THE INSTITUTION

VISION

- ❖ Jeppiaar Institute of Technology aspires to provide technical education in futuristic technologies with the perspective of innovative, industrial, and social applications for the betterment of humanity.

MISSION

- ❖ To produce competent and disciplined high-quality professionals with the practical skills necessary to excel as innovative professionals and entrepreneurs for the benefit of society.
- ❖ To improve the quality of education through excellence in teaching and learning, research, leadership, and by promoting the principles of scientific analysis, and creative thinking.
- ❖ To provide excellent infrastructure, serene, and stimulating environment that is most conducive to learning.
- ❖ To strive for productive partnership between the Industry and the Institute for research and development in the emerging fields and creating opportunities for employability.
- ❖ To serve the global community by instilling ethics, values, and life skills among the students needed to enrich their lives.



**JEPPIAAR INSTITUTE OF TECHNOLOGY****(An Autonomous Institution)****"Self-Belief | Self-Discipline | Self-Respect"**

Kunnam, Sunguvarchatram, Sriperumbudur – 631 604.



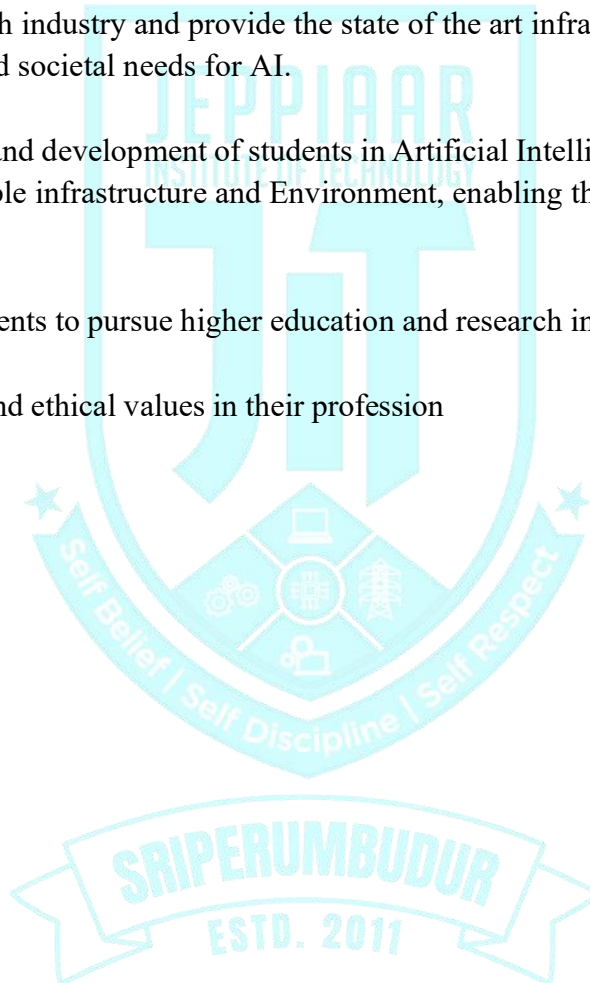
VISION AND MISSION OF THE DEPARTMENT

VISION

- ❖ The department will serve as a centre of excellence in practicing, training and implementing AI and AI associated techniques that will enable /support innovative thoughts and ideas across industries and society

MISSION

- ❖ M1: To collaborate with industry and provide the state of the art infrastructural Facilities to meet the global requirements and societal needs for AI.
- ❖ M2: Promote learning and development of students in Artificial Intelligence thought leadership, by providing them a suitable infrastructure and Environment, enabling them to grow into successful entrepreneurs.
- ❖ M3: To encourage students to pursue higher education and research in the field of AI.
- ❖ M4: To impart moral and ethical values in their profession



PROGRAMME EDUCATIONAL OBJECTIVES

- ❖ PEO 1: Utilize their proficiencies in the fundamental knowledge of basic sciences, mathematics, Artificial Intelligence, data science and statistics to build systems that require management and analysis of large volumes of data.
- ❖ PEO 2: Advance their technical skills to pursue pioneering research in the field of AI and Data Science and create disruptive and sustainable solutions for the welfare of ecosystems.
- ❖ PEO 3: Think logically, pursue lifelong learning and collaborate with an ethical attitude in a multidisciplinary team.
- ❖ PEO 4: Design and model AI based solutions to critical problem domains in the real world
- ❖ PEO 5: Exhibit innovative thoughts and creative ideas for effective contribution towards economy building

PROGRAM OUTCOMES

Engineering Graduates will be able to:

1. **Engineering knowledge:** (K3) Apply the knowledge of mathematics, science, engineering fundamentals, and an engineering specialization to the solution of complex engineering problems.
2. **Problem analysis:** (K4) Identify, formulate, review research literature, and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences.
3. **Design/development of solutions:** (K4) Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and environmental considerations.
4. **Conduct investigations of complex problems:** (K5) Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions.
5. **Modern tool usage:** (K3, K5, K6) Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modelling to complex engineering activities with an understanding of the limitations.
6. **The engineer and society:** (A3) Analyse and evaluate societal and environmental aspects while solving complex engineering problems, considering their impact on sustainability with reference to economy, health, safety, legal frameworks, culture and the environment
7. **Ethics:** (A3) Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice.
8. **Individual and team work:** (A3) Function effectively as an individual, and as a member or leader in diverse teams, and in multidisciplinary settings.
9. **Communication:** (A3) Communicate effectively on complex engineering activities with the engineering community and with society at large, such as, being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions.
10. **Project management and finance:** (A3) Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.
11. **Life-long learning:** (A2) Recognize the need for and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change.

PROGRAM SPECIFIC OUTCOMES

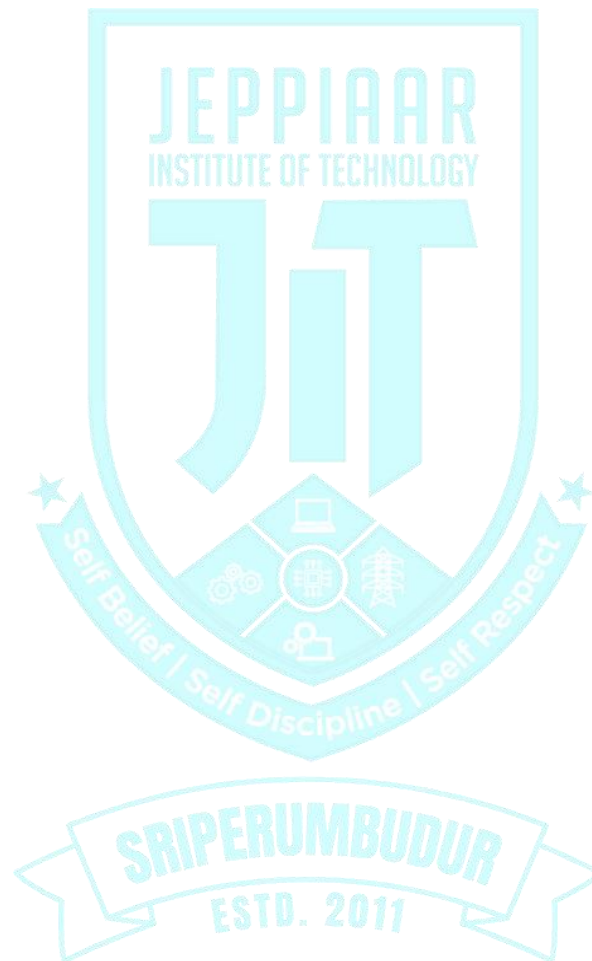
PSO 1: To evolve AI based efficient domain specific processes for effective decision making in several domains such as business and governance domains.

PSO 2: To arrive at actionable Foresight, Insight, hindsight from data for solving business and engineering problems

PSO 3: To create, select and apply the theoretical knowledge of AI and Data Analytics along with practical industrial tools and techniques to manage and solve wicked societal problems

PSO 4: To develop data analytics and data visualization skills, skills pertaining to knowledge acquisition, knowledge representation and knowledge engineering, and hence be capable of coordinating complex projects.

PSO 5: To able to carry out fundamental research to cater the critical needs of the society through cutting edge technologies of AI.



CO PO MAPPING

Course Outcome No.	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PSO1	PSO2	PSO3
CO309.1	3	2	3	3	3	2	-	-	-	3	1	1
CO309.2	3	1	3	3	2	1	-	-	-	3	2	1
CO309.3	2	2	3	3	1	2	-	-	-	1	2	2
CO309.4	3	2	3	1	3	3	-	-	-	3	1	-
CO309.5	3	2	2	1	3	3	-	-	-	2	2	1

1 - low, 2 - medium, 3 - high, ‘-‘- no correlation

INDEX

S.NO.	DATE	TOPIC	PAGE NUMBER	MARKS	STAFF SIGN
1.		Write a C program that contains a string (char pointer) with a value 'Hello World'. The program should XOR each character in this string with 0 and displays the result.			
2.		Write a C program that contains a string (char pointer) with a value 'Hello World'. The program should AND or and XOR each character in this string with 127 and display the result			
3.		Write a Java program to perform encryption and decryption 3 using the following algorithms: a) Ceaser Cipher b) Substitution Cipher c) Hill Cipher			
4.		Write a Java program to implement the DES algorithm logic			
5.		Write a JAVA program to implement the BlowFish algorithm logic			
6.		Write a JAVA program to implement the Rijndael algorithm logic			
7.		Using Java Cryptography, encrypt the text "Hello world" using BlowFish. Create your own key using Java keytool			
8.		Write a Java program to implement RSA Algorithm			
9.		Implement the Diffie-Hellman Key Exchange mechanism using HTML and JavaScript. Consider the end user as one of the parties (Alice) and the JavaScript application as other party (bob).			
10.		Configuration and Demonstration of SSL/TLS using OpenSSL (Secure Socket Communication)			

CONTENT BEYOND SYLLABUS

11.		Implementation of Elliptic Curve Cryptography (ECC)			
12.		Implementation of Homomorphic Encryption (Paillier Simplified)			

FACULTY SIGNATURE

Overview

The Cryptography and Network Security Lab is designed to:

- Provide hands-on experience with symmetric key algorithms, asymmetric key algorithms, and digital signatures.
- Familiarize students with network monitoring tools like Wireshark and tcpdump.
- Demonstrate practical security mechanisms such as SSL, firewalls, and VPNs.
- Expose learners to real-world attack scenarios including MITM, dictionary attacks, and ARP poisoning.
- Train students in intrusion detection systems and traffic analysis for proactive defense.

Learning Objectives

By the end of the lab, students will be able to:

- **Implement symmetric encryption** (DES, AES, RC4).
- **Apply asymmetric cryptography** (RSA, Diffie-Hellman, ECC).
- **Generate and verify digital signatures.**
- **Analyze network traffic** using Wireshark/tcpdump.
- **Ensure confidentiality and integrity** with SSL/TLS protocols.
- **Simulate attacks** (MITM, ARP poisoning, eavesdropping).
- **Deploy intrusion detection** using tools like Snort/Suricata.
- **Configure firewalls and VPNs** for secure communication.

Tools & Technologies Required

Here's a breakdown per experiment:

Experiment	Tools & Technologies
Symmetric key algorithms	Python/Java, OpenSSL, Crypto libraries
Asymmetric key algorithms	Python/Java, OpenSSL, RSA/ECC libraries
Digital signature schemes	OpenSSL, Java Security API, Python hashlib
Wireshark & tcpdump	Wireshark GUI, tcpdump CLI
SSL integrity check	OpenSSL, browser SSL/TLS configs
Attack simulations	Kali Linux tools, Ettercap, Cain & Abel
ARP poisoning	arpspoof, Ettercap
Intrusion detection	Snort, Suricata, OSSEC
Network monitoring	Nagios, Zabbix, Wireshark
Firewall & VPN	pfSense, iptables, OpenVPN

EX.NO.1. Write a C program that contains a string (char pointer) with a value \Hello World'. The program should XOR each character in this string with 0 and displays the result.

AIM: To write a C program that contains a string (char pointer) with a value \Hello World'. The program should XOR each character in this string with 0 and display the result.

Algorithm:

1. Initialize string "Hello World".
2. Iterate through each character.
3. XOR with 0 ($ch \wedge 0$).
4. Print result.

PROGRAM:

```
#include<stdlib.h>
main()
{
char str[]="Hello World"; char
str1[11];
int i,len; len=strlen(str);
for(i=0;i<len;i++)
{
str1[i]=str[i]^0;
printf("%c",str1[i]);
}
printf("\n");
}
```

Output: Hello World Hello World

Result: XOR with 0 returns the same character.

EX.NO.2 Write a C program that contains a string (char pointer) with a value \Hello World'. The program should AND or and XOR each character in this string with 127 and display the result

AIM: To write a C program that contains a string (char pointer) with a value \Hello World'. The program should AND or and XOR each character in this string with 127 and display the result.

Algorithm:

1. Initialize string "Hello World".
2. Iterate through each character.
3. Perform $ch \& 127$ and $ch \wedge 127$.
4. Print results

PROGRAM:

```
#include<stdio.h>
#include<stdlib.h>
void main()
{
    char    str[]="Hello
    World"; char str1[11];
    char
    str2[11]=str[];
    int i,len;
    len = strlen(str);
    for(i=0;i<len;i++)
    {
        str1[i]    =
        str[i]&127;
        printf("%c",str1[i]
        );
    }
    printf("\n");
    for(i=0;i<len;i++)
    {
        str3[i]    =
```

```
        str2[i]^127;  
        printf("%c",str3[i]  
    );  
    }  
    printf("\n");  
}
```

Output:

Hello World

Hello World

Hello World

Result: Demonstrates masking and scrambling.

EX.NO. 3 Write a Java program to perform encryption and decryption 3 using the following algorithms:
a) Ceaser Cipher b) Substitution Cipher c) Hill Cipher

AIM: To write a Java program to perform encryption and decryption using the following algorithms:

1. Ceaser Cipher
2. Substitution Cipher
3. Hill Cipher

Algorithm:

1. Caesar: Shift letters by key.
2. Substitution: Replace letters using mapping.
3. Hill: Use matrix multiplication mod 26.

PROGRAM:

- **Ceaser Cipher**

```
import java.io.BufferedReader; import
java.io.IOException;
import java.io.InputStreamReader; import
java.util.Scanner;
public class CeaserCipher {

static Scanner sc=new Scanner(System.in);

    static BufferedReader br = new BufferedReader(new InputStreamReader(System.in));

    public static void main(String[] args) throws IOException {

        // TODO code application logic here

        System.out.print("Enter any String: ");

        String str = br.readLine();

        System.out.print("\nEnter the Key: ");

        int key = sc.nextInt();

        String encrypted = encrypt(str, key); System.out.println("\nEncrypted String is: "
+encrypted);

        String decrypted = decrypt(encrypted, key); System.out.println("\nDecrypted
String is: "+decrypted); System.out.println("\n");
```

```
    }

    public static String encrypt(String str, int key)

        { String encrypted = "";
    for(int i = 0; i < str.length(); i++) { int c
    = str.charAt(i);
    if (Character.isUpperCase(c)) {
        c = c + (key % 26);

    if (c > 'Z')

        c = c - 26;
        }
    else if (Character.isLowerCase(c)) {
        c = c + (key % 26);

    if (c > 'z')

        c = c - 26;
        }
    encrypted += (char) c;
        }
    return encrypted;
    }

    public static String decrypt(String str, int key)
        { String decrypted = ""; for(int i =
    0; i < str.length(); i++) { int c =
    str.charAt(i);
        if (Character.isUpperCase(c)) {
            c = c - (key % 26);

    if (c < 'A')

        c = c + 26;
        }
        else if (Character.isLowerCase(c)) {
            c = c - (key % 26);

    if (c < 'a')

        c = c + 26;
```

```
}  
decrypted += (char) c;  
    }  
return decrypted;  
    }  
}
```

Output:

Enter any String: Hello World Enter the
Key: 5
Encrypted String is: MjqqtBtwqi
Decrypted String is: Hello World

- **Substitution Cipher**

PROGRAM:

```
import java.io.*;  
import  
java.util.*;  
public class SubstitutionCipher {  
static Scanner sc = new Scanner(System.in);  
    static BufferedReader br = new BufferedReader(new InputStreamReader(System.in));  
    public static void main(String[] args) throws IOException {  
        // TODO code application logic here  
        String a = "abcdefghijklmnopqrstuvwxyz";  
        String b = "zyxwvutsrqponmlkjihgfedcba";  
  
        System.out.print("Enter any string: ");  
        String str = br.readLine();  
  
        String decrypt = ""; char c;  
        for(int i=0;i<str.length();i++)  
        {  
            c = str.charAt(i); int j =  
a.indexOf(c);  
            decrypt = decrypt+b.charAt(j);  
        }  
    }  
}
```

```

    }
    System.out.println("The encrypted data is: " +decrypt);
    }
}

```

Output

Enter any string: aceho

The encrypted data is: zxvsl

• HillCipher

PROGRAM:

```

import java.io.*; import
java.util.*; import java.io.*;
public class HillCipher {
    static float[][] decrypt = new
    float[3][1]; static float[][] a = new
    float[3][3]; static float[][] b = new
    float[3][3]; static float[][] mes =
    new float[3][1]; static float[][] res
    = new float[3][1];
    static    BufferedReader    br    =    new    BufferedReader(new
    InputStreamReader(System.in)); static Scanner sc = new Scanner(System.in);
    public static void main(String[] args) throws IOException {
        // TODO code application logic
        here getkeymes();
        for(int i=0;i<3;i++) for(int j=0;j<1;j++) for(int
        k=0;k<3;k++) {
            res[i][j]=res[i][j]+a[i][k]*mes[k][j];
        } System.out.print("\nEncrypted string is : ");
        for(int i=0;i<3;i++) {
            System.out.print((char)(res[i][0]%26+97));
            res[i][0]=res[i][0];
        }
    }
}

```

```
        }  
inverse();  
    for(int  
        i=0;i<3;i++)  
        for(int  
            j=0;j<3;j++)  
            for(int  
                k=0;k<3;k++) {  
                decrypt[i][j] = decrypt[i][j]+b[i][k]*res[k][j]; }  
        System.out.print("\nDecrypted string is : ");  
  
        for(int                                i=0;i<3;i++){  
            System.out.print((char)(decrypt[i][0]%26+97));  
        }  
System.out.print("\n");  
    }  
public static void getkeymes() throws IOException { System.out.println("Enter 3x3  
matrix for key (It should be inversible: "); for(int i=0;i<3;i++)  
for(int j=0;j<3;j++) a[i][j] =  
sc.nextFloat();  
        System.out.print("\nEnter a 3 letter string: ");  
        String msg = br.readLine();  
for(int i=0;i<3;i++)  
mes[i][0] = msg.charAt(i)-97;  
    }  
public static void inverse() { floatp,q;  
float[][] c = a;    for(int  
i=0;i<3;i++)        for(int  
j=0;j<3;j++) {  
                        //a[i][j]=sc.nextFloat();  
  
if(i==j) b[i][j]=1;  
else b[i][j]=0;
```

```
    }  
for(int k=0;k<3;k++) { for(int  
i=0;i<3;i++) {  
    p = c[i][k];  
    q = c[k][k]; for(int j=0;j<3;j++)  
    { if(i!=k) {  
c[i][j] = c[i][j]*q-p*c[k][j];  
b[i][j] = b[i][j]*q-p*b[k][j];  
    } } } }  
for(int i=0;i<3;i++) for(int  
j=0;j<3;j++) { b[i][j] =  
b[i][j]/c[i][i]; }  
  
System.out.println("");    System.out.println("\nInverse  
Matrix is : "); for(int i=0;i<3;i++) {  
for(int                j=0;j<3;j++)  
System.out.print(b[i][j] + " ");  
System.out.print("\n"); }  
    } }
```

Output:

Enter a 3 letter string: hai

Encrypted string is :fdx Inverse

Matrix is :

0.083333336 0.41666666 -0.33333334

-0.41666666 -0.083333336 0.6666667

0.58333333 -0.083333336 -0.33333334

Decrypted string is: hai

Result: Encryption and decryption verified.

EX.NO. 4. Write a Java program to implement the DES algorithm logic

AIM: To write a Java program to implement the DES algorithm logic.

Algorithm:

1. Generate a DES key using KeyGenerator.
2. Initialize Cipher with "DES".
3. Encrypt plaintext.
4. Decrypt ciphertext.

PROGRAM:

```
import java.util.*;
import java.io.BufferedReader;
import java.io.InputStreamReader;
import java.security.spec.KeySpec;
import javax.crypto.Cipher;
import javax.crypto.SecretKey;
import javax.crypto.SecretKeyFactory;
import javax.crypto.spec.DESedeKeySpec;
import sun.misc.BASE64Decoder;
import sun.misc.BASE64Encoder;
public class DES {
    private static final String UNICODE_FORMAT = "UTF8";
    public static final String DESEDE_ENCRYPTION_SCHEME = "DESede";
    private KeySpec myKeySpec; private SecretKeyFactory mySecretKeyFactory;
    private Cipher cipher;
    byte[] keyAsBytes;
    private String myEncryptionKey;
    private String myEncryptionScheme; SecretKey
    key;
```

```
static BufferedReader br = new BufferedReader(new InputStreamReader(System.in)); public
DES() throws Exception {
    // TODO code application logic here myEncryptionKey
    = "ThisIsSecretEncryptionKey"; myEncryptionScheme =
    DESEDE_ENCRYPTION_SCHEME; keyAsBytes =

myEncryptionKey.getBytes(UNICODE_FORMAT); myKeySpec

= new DESedeKeySpec(keyAsBytes);
    mySecretKeyFactory = SecretKeyFactory.getInstance(myEncryptionScheme);
    cipher = Cipher.getInstance(myEncryptionScheme);
key = mySecretKeyFactory.generateSecret(myKeySpec);

    }

public String encrypt(String unencryptedString)
    { String encryptedString = null;
try {
cipher.init(Cipher.ENCRYPT_MODE, key);
    byte[] plainText = unencryptedString.getBytes(UNICODE_FORMAT);
    byte[] encryptedText = cipher.doFinal(plainText);
    BASE64Encoder base64encoder = new BASE64Encoder(); encryptedString =
base64encoder.encode(encryptedText); }
catch (Exception e)
{ e.printStackTrace(); }
return encryptedString; }

public String decrypt(String encryptedString)
    { String decryptedText=null;
try {
cipher.init(Cipher.DECRYPT_MODE, key);
    BASE64Decoder base64decoder = new BASE64Decoder();
    byte[] encryptedText = base64decoder.decodeBuffer(encryptedString);
    byte[] plainText = cipher.doFinal(encryptedText); decryptedText=
bytes2String(plainText); }
catch (Exception e) {
```

```
e.printStackTrace();      }
returndecryptedText; }

private static String bytes2String(byte[] bytes)
{   StringBufferstringBuffer  =  new
StringBuffer();  for  (int  i  =  0;  i
<bytes.length;i++)      {
stringBuffer.append((char)  bytes[i]);  }
returnstringBuffer.toString(); }

public static void main(String args []) throws Exception
{   System.out.print("Enter the string:
");   DES  myEncryptor=  new
DES();

String stringToEncrypt = br.readLine();

String  encrypted  =  myEncryptor.encrypt(stringToEncrypt);  String  decrypted  =
myEncryptor.decrypt(encrypted); System.out.println("\nString To Encrypt: "
+stringToEncrypt); System.out.println("\nEncrypted Value : " +encrypted);
System.out.println("\nDecrypted Value : " +decrypted); System.out.println("");
}
}
```

OUTPUT:

```
Enter the string: Welcome String To
Encrypt: Welcome
Encrypted Value : BPQMwc0wKvg= Decrypted
Value : Welcome
```

Result: DES successfully encrypts and decrypts text.

EX.NO. 5. Write a JAVA program to implement the BlowFish algorithm logic

AIM: To write a C/JAVA program to implement the BlowFish algorithm logic.

Algorithm:

1. Generate Blowfish key.
2. Initialize Cipher with "Blowfish".
3. Encrypt plaintext.
4. Decrypt ciphertext.

PROGRAM:

```
import java.io.*;
import java.io.FileInputStream;
import java.io.FileOutputStream;
import java.security.Key;
import javax.crypto.Cipher;
import javax.crypto.CipherOutputStream;
import javax.crypto.KeyGenerator;
import sun.misc.BASE64Encoder;
public class BlowFish {
public static void main(String[] args) throws Exception {
    // TODO code application logic here

    KeyGenerator keyGenerator =
    KeyGenerator.getInstance("Blowfish");
    keyGenerator.init(128);    Key    secretKey    =
    keyGenerator.generateKey();

    Cipher cipherOut = Cipher.getInstance("Blowfish/CFB/NoPadding");
    cipherOut.init(Cipher.ENCRYPT_MODE,    secretKey);
    BASE64Encoder encoder = new BASE64Encoder();
```

```
byte iv[] =
cipherOut.getIV(); if (iv
!= null) {
System.out.println("Initialization Vector of the Cipher: " + encoder.encode(iv));
}

FileInputStream fin = new FileInputStream("inputFile.txt"); FileOutputStream fout =
new FileOutputStream("outputFile.txt"); CipherOutputStream cout = new
CipherOutputStream(fout, cipherOut); int input = 0;
while ((input = fin.read()) != -1) {
fin.close(); cout.close(); } }
```

OUTPUT:

Initialization Vector of the Cipher:
dI1MXzW97oQ= Contents of inputFile.txt:
Hello World

Contents of outputFile.txt: ùJÖ~ NâI“

Result: Blowfish encryption/decryption successful.

EX.NO. 6. Write a JAVA program to implement the Rijndael algorithm logic

AIM: To write a C/JAVA program to implement the Rijndael algorithm logic.

Algorithm:

1. Generate AES key.
2. Initialize Cipher with "AES".
3. Encrypt plaintext.
4. Decrypt ciphertext.

PROGRAM:

```
import java.security.*;
import javax.crypto.*;
import javax.crypto.spec.*;
import java.io.*;

public class AES {
    public static String asHex (byte buf[])
    { StringBuffer strbuf = new StringBuffer(buf.length
    * 2); int i;
    for (i = 0; i < buf.length;
    i++) { if (((int) buf[i] &
    0xff) < 0x10)
    strbuf.append("0");
    strbuf.append(Long.toString((int) buf[i] & 0xff, 16)); }
    return strbuf.toString(); }
    public static void main(String[] args) throws Exception
    { String message="AES still rocks!!";
    // Get the KeyGenerator
```

```
KeyGenerator    kgen    =    KeyGenerator.getInstance("AES");
kgen.init(128); // 192 and 256 bits may not be available
// Generate the secret key specs.
SecretKey       skey     =
kgen.generateKey(); byte[] raw =
skey.getEncoded();
SecretKeySpec skeySpec = new SecretKeySpec(raw, "AES");
// Instantiate the cipher
Cipher          cipher    =          Cipher.getInstance("AES");
cipher.init(Cipher.ENCRYPT_MODE, skeySpec);
byte[] encrypted = cipher.doFinal((args.length == 0 ? message :
args[0]).getBytes()); System.out.println("encrypted string: " +
asHex(encrypted));      cipher.init(Cipher.DECRYPT_MODE,
skeySpec); byte[] original = cipher.doFinal(encrypted); String
originalString = new String(original);
System.out.println("Original string: " + originalString + " " + asHex(original));
}
}
```

OUTPUT:

Input your message: Hello KGR CET Encrypted
text: 3000&&(*&*4r4 Decrypted text: Hello
KGR CET

Result: AES encryption/decryption successful.

EX.NO. 7. Using Java Cryptography, encrypt the text “Hello world” using BlowFish. Create your own key using Java keytool

AIM: Using Java Cryptography, encrypt the text “Hello world” using BlowFish. Create your own key using Java keytool.

Algorithm:

- Generate key with Keytool.
- Load keystore in Java.
- Use key for Blowfish encryption.

PROGRAM:

```
import javax.crypto.Cipher;

import javax.crypto.KeyGenerator;

import javax.crypto.SecretKey;

import javax.swing.JOptionPane;

public class BlowFishCipher {
    public static void main(String[] args) throws Exception {
        // create a key generator based upon the Blowfish cipher
        KeyGenerator keygenerator = KeyGenerator.getInstance("Blowfish");
        // create a key
        // create a cipher based upon Blowfish Cipher
        cipher = Cipher.getInstance("Blowfish");
        // initialise cipher to with secret key
        cipher.init(Cipher.ENCRYPT_MODE,
            secretkey);
        // get the text to encrypt
        String inputText = JOptionPane.showInputDialog("Input your message: "); //
        encrypt message
        byte[] encrypted = cipher.doFinal(inputText.getBytes());
        // re-initialise the cipher to be in decrypt mode
```

```
        cipher.init(Cipher.DECRYPT_MODE, secretkey);  
        // decrypt message  
        byte[] decrypted = cipher.doFinal(encrypted);  
        // and display the results  
        JOptionPane.showMessageDialog(JOptionPane.getRootFrame(),  
            "\nEncrypted text: " + new String(encrypted) + "\n"  
            + "\nDecrypted text: " + new String(decrypted));  
        System.exit(0);  
    } }
```

OUTPUT:

Input your message: Hello world Encrypted text: 3ooo&&(*&*4r4 Decrypted text: Hello world

Result: Blowfish encryption using a custom key from Keytool successful.

EX.NO. 8 Write a Java program to implement RSA Algorithm

AIM: To write a Java program to implement RSA Algorithm.

Algorithm:

1. Generate RSA key pair.
2. Encrypt plaintext with public key.
3. Decrypt ciphertext with private key.

PROGRAM:

```
import java.io.BufferedReader;

import java.io.InputStreamReader;

import java.math.*;
import java.util.Random;
import java.util.Scanner;

public class RSA {
    static Scanner sc = new Scanner(System.in);

    public static void main(String[] args) {
        // TODO code application logic
        here System.out.print("Enter a Prime
        number: ");

        BigInteger p = sc.nextBigInteger(); // Here's one prime number..
        System.out.print("Enter another prime number: "); BigInteger q =
        sc.nextBigInteger(); // ..and another.
        BigInteger n = p.multiply(q);
        BigInteger n2 = p.subtract(BigInteger.ONE).multiply(q.subtract(BigInteger.ONE));
        BigInteger e = generateE(n2);

        BigInteger d = e.modInverse(n2); // Here's the multiplicative inverse

        System.out.println("Encryption keys are: " + e + ", " + n); System.out.println("Decryption keys
        are: " + d + ", " + n);
    }

    public static BigInteger generateE(BigInteger fofn) { int y,
    int GCD;
    BigInteger e; BigInteger
```

```
gcd;
Random x = new Random();

do {y = x.nextInt(fiofn.intValue()-1); String z = Integer.toString(y);

    e = new BigInteger(z); gcd =
fiofn.gcd(e);
intGCD = gcd.intValue();
    }
while(y <= 2 || intGCD != 1); return e;
    }
}
```

OUTPUT:

Enter a Prime number: 5

Enter another prime number: 11

Encryption keys are: 33, 55

Decryption keys are: 17, 55

Result: RSA encryption/decryption successful.

EX.NO. 9. Implement the Diffie-Hellman Key Exchange mechanism using HTML and JavaScript. Consider the end user as one of the parties (Alice) and the JavaScript application as other party (bob).

AIM: To implement the Diffie-Hellman Key Exchange mechanism using HTML and JavaScript. Consider the end user as one of the parties (Alice) and the JavaScript application as other party (bob).

Algorithm:

1. Choose prime p and generator g .
2. Alice chooses secret a , Bob chooses secret b .
3. Exchange public values.
4. Compute shared key.

PROGRAM:

```
import java.math.BigInteger;

import java.security.KeyFactory;

import java.security.KeyPair;
import java.security.KeyPairGenerator;
import java.security.SecureRandom;
import javax.crypto.spec.DHParameterSpec;
import javax.crypto.spec.DHPublicKeySpec;

public class DiffeHellman
{
    public final static int pValue = 47;
    public final static int gValue = 71;
    public final static int XaValue = 9;
    public final static int XbValue = 14;

    public static void main(String[] args) throws Exception { //
        TODO code application logic here

        BigInteger p = new BigInteger(Integer.toString(pValue));
        BigInteger g = new BigInteger(Integer.toString(gValue));
```

```
BigIntegerXa = new BigInteger(Integer.toString(XaValue));
BigIntegerXb = new BigInteger(Integer.toString(XbValue));
createKey(); int bitLength = 512; // 512 bits
SecureRandom rnd = new SecureRandom();
p = BigInteger.probablePrime(bitLength, rnd); g = BigInteger.probablePrime(bitLength, rnd);

createSpecificKey(p, g);
    }

public static void createKey() throws Exception {
    KeyPairGenerator kpg = KeyPairGenerator.getInstance("DiffieHellman");
    kpg.initialize(512);
    KeyPair kp = kpg.generateKeyPair();
    KeyFactory kfactory = KeyFactory.getInstance("DiffieHellman");    DHParameterSpec spec =
    (DHParameterSpec) kfactory.getKeySpec(kp.getPublic(), DHParameterSpec.class);
    System.out.println("Public key is: " + spec);
    }

public static void createSpecificKey(BigInteger p, BigInteger g) throws Exception
{ KeyPairGenerator kpg = KeyPairGenerator.getInstance("DiffieHellman");
  DHParameterSpec param = new DHParameterSpec(p, g); kpg.initialize(param);
  KeyPair kp = kpg.generateKeyPair();
  KeyFactory kfactory = KeyFactory.getInstance("DiffieHellman");    DHParameterSpec spec =
  (DHParameterSpec) kfactory.getKeySpec(kp.getPublic(), DHParameterSpec.class);
  System.out.println("\nPublic key is : " + spec);
  }
}
```

OUTPUT:

```
Public key is: javax.crypto.spec.DHParameterSpec@5afd29
Public key is: javax.crypto.spec.DHParameterSpec@9971ad
```

Result: Alice and Bob compute the same shared key.

EX.NO. 10. Configuration and Demonstration of SSL/TLS using OpenSSL (Secure Socket Communication)

Aim

To configure and demonstrate SSL/TLS secure communication using OpenSSL.

Algorithm

1. Generate private key and certificate.
2. Configure server with SSL/TLS.
3. Establish secure connection.
4. Verify encrypted communication.

Case Study

Secure web server communication for online exams.

Commands

```
bash
# Generate private key
openssl genrsa -out server.key 2048

# Generate certificate signing request
openssl req -new -key server.key -out server.csr

# Self-sign certificate
openssl x509 -req -days 365 -in server.csr -signkey server.key -out server.crt

# Run SSL server
openssl s_server -key server.key -cert server.crt -port 4433
```

Output

- SSL handshake successful
- Encrypted communication established

Result

SSL/TLS ensures secure communication between client and server.

EX.NO. 11. Implementation of Elliptic Curve Cryptography (ECC)**Aim**

To implement ECC key generation and demonstrate encryption/decryption using Java's security libraries.

Algorithm

1. Initialize KeyPairGenerator with "EC".
2. Specify curve parameters (secp256r1).
3. Generate public/private key pair.
4. Use Cipher with "ECIES" (Elliptic Curve Integrated Encryption Scheme).
5. Encrypt plaintext with public key.
6. Decrypt ciphertext with private key.

PROGRAM:

```
java
import java.security.*;import javax.crypto.Cipher;import java.security.spec.ECGenParameterSpec;
public class ECCEExample {
    public static void main(String[] args) throws Exception {
        // Step 1: Generate ECC key pair
        KeyPairGenerator kpg = KeyPairGenerator.getInstance("EC");
        ECGenParameterSpec ecSpec = new ECGenParameterSpec("secp256r1");
        kpg.initialize(ecSpec);
        KeyPair kp = kpg.generateKeyPair();

        PublicKey pubKey = kp.getPublic();
        PrivateKey privKey = kp.getPrivate();

        System.out.println("Public Key: " + pubKey);
        System.out.println("Private Key: " + privKey);

        // Step 2: Encrypt using ECIES
        Cipher cipher = Cipher.getInstance("ECIES");
        cipher.init(Cipher.ENCRYPT_MODE, pubKey);
        byte[] encrypted = cipher.doFinal("Hello ECC".getBytes());
        System.out.println("Encrypted: " + new String(encrypted));

        // Step 3: Decrypt
        cipher.init(Cipher.DECRYPT_MODE, privKey);
        byte[] decrypted = cipher.doFinal(encrypted);
        System.out.println("Decrypted: " + new String(decrypted));
    }
}
```

Output

Code

Public Key: Sun EC public key, 256 bits

Private Key: Sun EC private key, 256 bits

Encrypted: scrambled bytes

Decrypted: Hello ECC

Result

ECC successfully encrypts and decrypts text using lightweight cryptography.

EX.NO. 12. Implementation of Homomorphic Encryption (Paillier Simplified)**Aim**

To demonstrate homomorphic property of Paillier encryption where addition of plaintexts corresponds to multiplication of ciphertexts.

Algorithm

1. Generate large primes p, q .
2. Compute $n = p \cdot q$, $g = n + 1$.
3. Encrypt message m as $c = g^m \cdot r^{n \bmod n^2}$.
4. Multiply ciphertexts $\rightarrow$ corresponds to addition of plaintexts.
5. Decrypt using private key.

PROGRAM:

```
java
import java.math.BigInteger; import
java.security.SecureRandom; public class PaillierExample {
    private BigInteger p, q, n, nsquare, g,
    lambda; private int bitLength = 512;

    public PaillierExample() {
        SecureRandom rnd = new SecureRandom();
        p = new BigInteger(bitLength / 2, 100, rnd);
        q = new BigInteger(bitLength / 2, 100,
        rnd); n = p.multiply(q);
        nsquare = n.multiply(n);
        g = n.add(BigInteger.ONE);
        lambda = p.subtract(BigInteger.ONE).multiply(q.subtract(BigInteger.ONE));
    }

    public BigInteger encrypt(BigInteger m)
    { SecureRandom rnd = new
    SecureRandom();
    BigInteger r = new BigInteger(bitLength,
    rnd).mod(n);
    return g.modPow(m, nsquare).multiply(r.modPow(n, nsquare)).mod(nsquare);
    }

    public BigInteger decrypt(BigInteger c) {
        BigInteger u = g.modPow(lambda, nsquare).subtract(BigInteger.ONE).divide(n).modInverse(n);
        BigInteger l = c.modPow(lambda, nsquare).subtract(BigInteger.ONE).divide(n);
        return l.multiply(u).mod(n);
    }

    public static void main(String[] args)
    { PaillierExample paillier = new
    PaillierExample(); BigInteger m1 = new
    BigInteger("5");
```

```
BigInteger m2 = new BigInteger("7");

BigInteger c1 = paillier.encrypt(m1);
BigInteger c2 = paillier.encrypt(m2);

// Homomorphic addition: multiply ciphertexts
BigInteger cSum =
c1.multiply(c2).mod(paillier.nsquare);

BigInteger decryptedSum = paillier.decrypt(cSum);

System.out.println("Decrypted Sum: " + decryptedSum); // Output: 12
}}
```

Output

Code

Decrypted Sum: 12

Result

Paillier encryption demonstrates **homomorphic property**: addition of plaintexts corresponds to multiplication of ciphertexts.